Normalization and Functional Dependency

Relational Model in DBMS

- A **Relational Database management System**(RDBMS) is a database management system based on the relational model introduced by E.F Codd. In relational model, data is stored in **relations**(tables) and is represented in form of **tuples**(rows).

- **RDBMS** is used to manage Relational database.

- **Relational database** is a collection of organized set of tables related to each other, and from which data can be accessed easily. Relational Database is the most commonly used database these days.

RDBMS: What is Table ?

In Relational database model, a **table** is a collection of data elements organised in terms of rows and columns. A table is also considered as a convenient representation of **relations**. But a table can have duplicate row of data while a true **relation** cannot have duplicate data. Table is the most simplest form of data storage. Below is an example of an Employee table.

ID	Name	Age	Salary
1	Adam	34	13000
2	Alex	28	15000
3	Stuart	20	18000
4	Ross	42	19020

RDBMS : What is a Tuple?

A single entry in a table is called a **Tuple** or **Record** or **Row**. A **tuple** in a table represents a set of related data. For example, the above **Employee** table has 4 tuples/records/rows.

Following is an example of single record or tuple.

1	Adam	34	13000

RDBMS : What is an Attribute?

A table consists of several records(row), each record can be broken down into several smaller parts of data known as **Attributes**. The above **Employee** table consist of four attributes, **ID**, **Name**, **Age** and **Salary**.

Attribute Domain

When an attribute is defined in a relation(table), it is defined to hold only a certain type of values, which is known as **Attribute Domain**.

Hence, the attribute **Name** will hold the name of employee for every tuple. If we save employee's address there, it will be violation of the Relational database model.

Name
Adam
Alex
Stuart - 9/401, OC Street, Amsterdam
Ross

What is a Relation Schema?

A relation schema describes the structure of the relation, with the name of the relation(name of table), its attributes and their names and type.

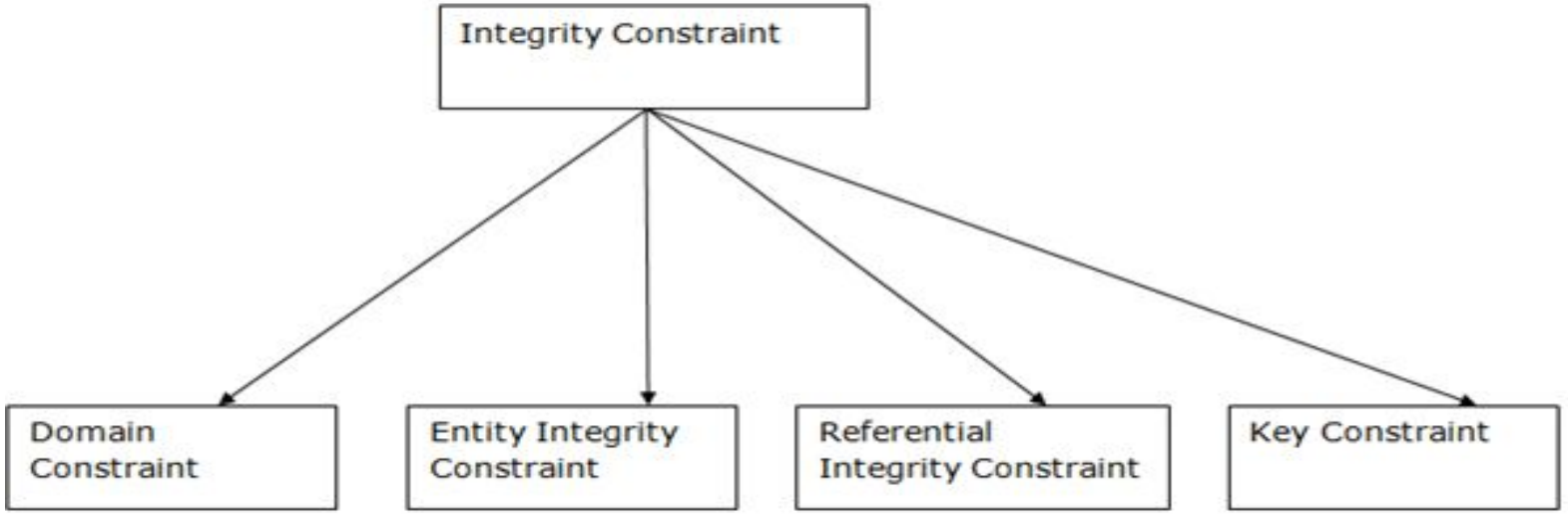
What is a Relation Key?

A relation key is an attribute which can uniquely identify a particular tuple(row) in a relation(table).

Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

Integrity Constraints



Integrity Constraints

1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed. Because AGE is an integer attribute

Integrity Constraints

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

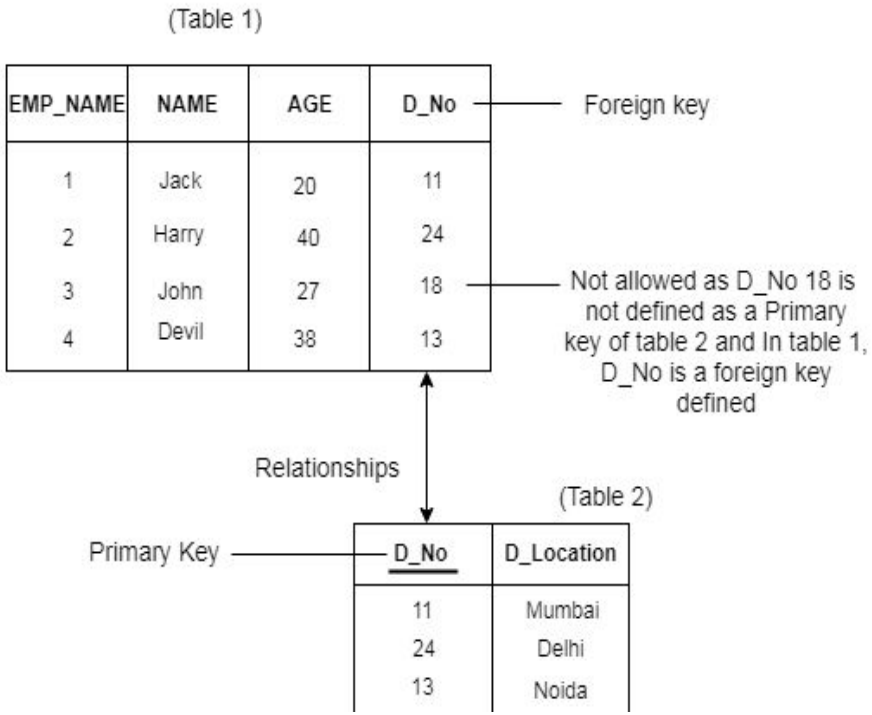
Not allowed as primary key can't contain a NULL value

Integrity Constraints

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:



Integrity Constraints

4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1002	Morgan	8th	22

Not allowed. Because all row must be unique



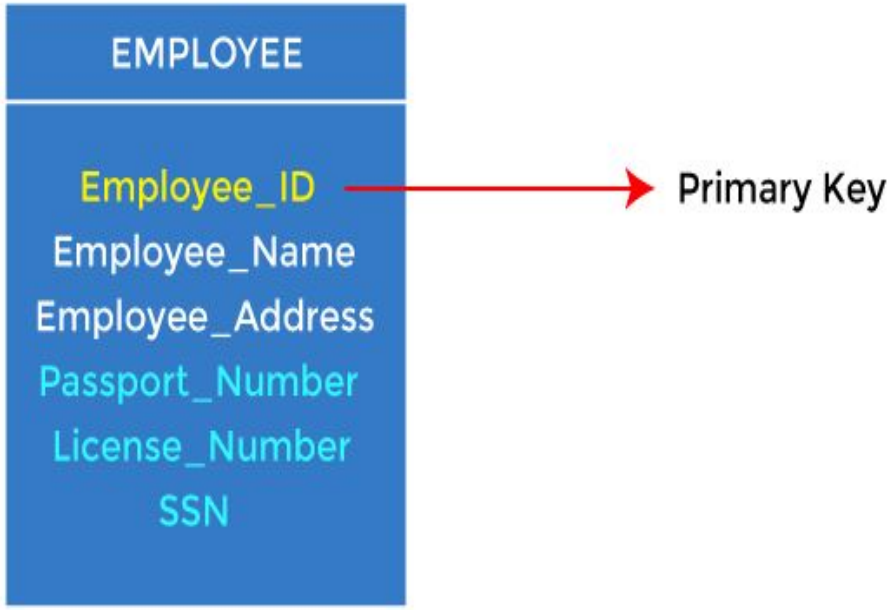
Types of Keys





1. Primary key

- It is the first key used to identify one and only one instance of an entity uniquely. An entity can contain multiple keys, as we saw in the PERSON table. The key which is most suitable from those lists becomes a primary key.
- In the EMPLOYEE table, ID can be the primary key since it is unique for each employee. In the EMPLOYEE table, we can even select License_Number and Passport_Number as primary keys since they are also unique.
- For each entity, the primary key selection is based on requirements and developers.

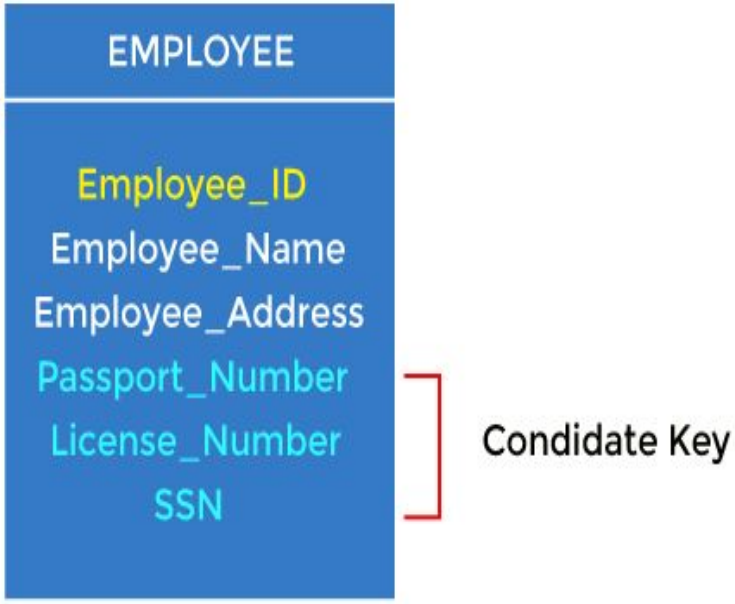




2. Candidate key

- A candidate key is an attribute or set of attributes that can uniquely identify a tuple.
- Except for the primary key, the remaining attributes are considered a candidate key. The candidate keys are as strong as the primary key.

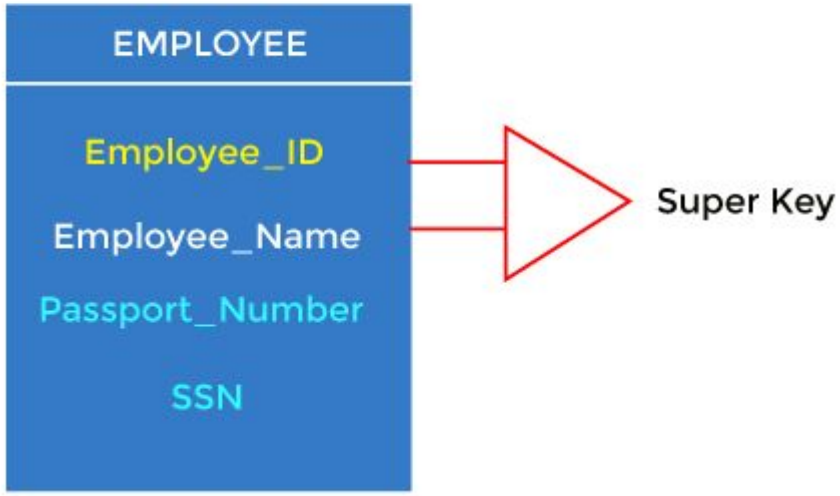
For example: In the EMPLOYEE table, id is best suited for the primary key. The rest of the attributes, like SSN, Passport_Number, License_Number, etc., are considered a candidate key.





3. Super Key

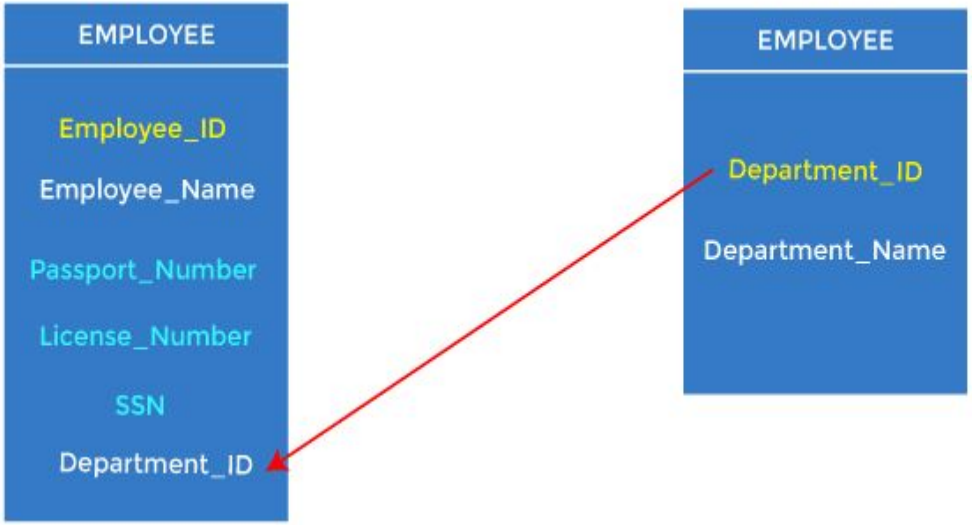
Super key is an attribute set that can uniquely identify a tuple. A super key is a superset of a candidate key.



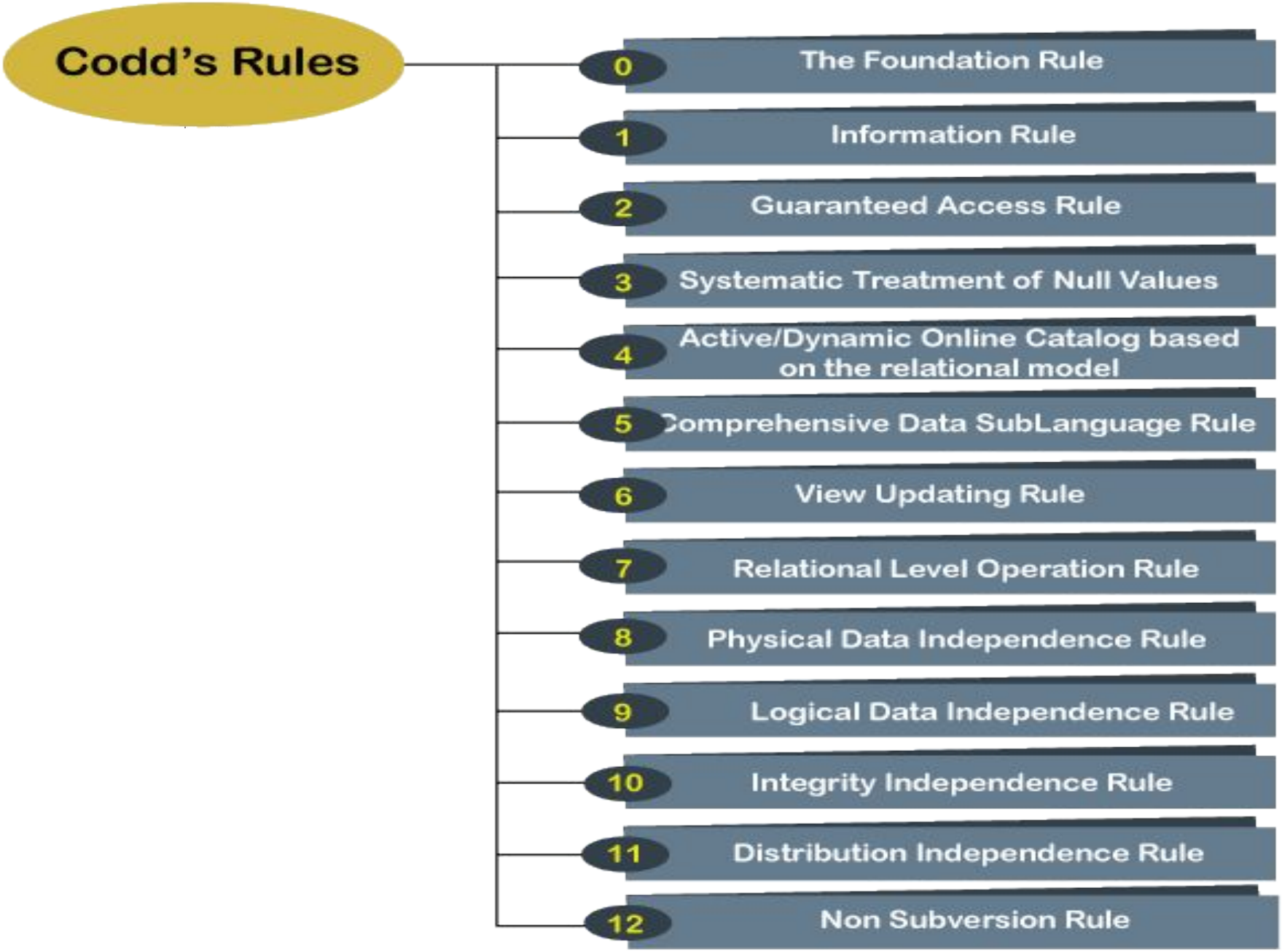


4. Foreign key

- Foreign keys are the column of the table used to point to the primary key of another table.
- Every employee works in a specific department in a company, and employee and department are two different entities. So we can't store the department's information in the employee table. That's why we link these two tables through the primary key of one table.
- We add the primary key of the DEPARTMENT table, Department_Id, as a new attribute in the EMPLOYEE table.
- In the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.



12 Codd's Rules



Rule 1: Information Rule

The data stored in a database, may it be user data or metadata, must be a value of some table cell. Everything in a database must be stored in a table format.

Rule 2: Guaranteed Access Rule

Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value). No other means, such as pointers, can be used to access data.

Rule 3: Systematic Treatment of NULL Values

The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following – data is missing, data is not known, or data is not applicable.

Rule 4: Active Online Catalog

The structure description of the entire database must be stored in an online catalog, known as **data dictionary**, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself.

Rule 5: Comprehensive Data Sub-Language Rule

A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations. This language can be used directly or by means of some application. If the database allows access to data without any help of this language, then it is considered as a violation.

Rule 6: View Updating Rule

All the views of a database, which can theoretically be updated, must also be updatable by the system.

Rule 7: High-Level Insert, Update, and Delete Rule

A database must support high-level insertion, updation, and deletion. This must not be limited to a single row, that is, it must also support union, intersection and minus operations to yield sets of data records.

Rule 8: Physical Data Independence

The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.

Rule 9: Logical Data Independence

The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.

Rule 10: Integrity Independence

A database must be independent of the application that uses it. All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface.

Rule 11: Distribution Independence

The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only. This rule has been regarded as the foundation of distributed database systems.

Rule 12: Non-Subversion Rule

If a system has an interface that provides access to low-level records, then the interface must not be able to subvert the system and bypass security and integrity constraints.

Features of good relational designs

1. Normalization

- **Reduction of Redundancy**: Data is organized to minimize duplication.
- **Avoidance of Anomalies**: Ensures that insertions, deletions, and updates do not lead to inconsistencies.

2. Clear Structure

- **Well-Defined Tables**: Each table should represent a single entity or concept.
- **Appropriate Use of Keys**: Primary keys uniquely identify records, while foreign keys maintain relationships between tables.

3. Data Integrity

- **Constraints**: Implement constraints (e.g., primary key, foreign key, unique, check) to enforce data integrity and consistency.
- **Referential Integrity**: Foreign keys ensure that relationships between tables remain valid.

Features of good relational designs

4. Flexibility and Scalability

- **Adaptable Schema**: The design should allow for easy modification, such as adding new tables or fields without significant disruption.
- **Efficient Querying**: The structure should support complex queries without performance degradation.

5. Performance Optimization

- **Indexing**: Use indexes on frequently queried columns to speed up search operations.
- **Denormalization (when appropriate)**: Sometimes, controlled redundancy can improve performance for specific queries.

6. Clear Naming Conventions

- **Descriptive Names**: Tables and columns should have meaningful names that clearly indicate their purpose.
- **Consistency**: Use consistent naming conventions throughout the database to enhance readability.

Features of good relational designs

7. Documentation

- **Schema Documentation**: Maintain clear documentation of the database structure, including tables, relationships, and constraints.
- **Change Logs**: Keep track of modifications to the schema and design decisions for future reference.

8. Security

- **Access Controls**: Implement user roles and permissions to restrict access to sensitive data.
- **Data Protection**: Ensure that data is protected against unauthorized access and breaches.

9. Support for Transactions

- **ACID Properties**: Ensure that transactions are Atomic, Consistent, Isolated, and Durable to maintain data integrity in multi-user environments.

Normalization in DBMS

- DBMS Normalization is a systematic approach to decompose (break down) tables to eliminate data redundancy(repetition) and undesirable characteristics like :-
- Insertion anomaly in DBMS,
- Update anomaly in DBMS, and
- Delete anomaly in DBMS.

- It is a multi-step process that puts data into tabular form, removes duplicate data, and set up the relationship between tables.

Why we need Normalization in DBMS ?

Normalization is required for,

- Eliminating redundant(useless) data, therefore handling **data integrity**, because if data is repeated it increases the chances of inconsistent data.
- Normalization helps in keeping **data consistent** by storing the data in one table and referencing it everywhere else
- Storage optimization although that is not an issue these days because Database storage is cheap.
- Breaking down large tables into smaller tables with relationships, so it makes the database structure more scalable and adaptable.
- Ensuring data dependencies make sense i.e. data is logically stored.

Problems without Normalization in DBMS

If a table is not properly normalized and has data redundancy(repetition) then it will not only **eat up extra memory space** but will also make it difficult for you to handle and update the data in the database, without losing data.

Insertion, Updation, and Deletion Anomalies are very frequent if the database is not normalized.

To understand these anomalies let us take an example of a **Student** table.

rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

In the table above, we have *data for four Computer Sci. students.*

1. Insertion Anomaly in DBMS

- Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as **NULL**.
- Also, if we have to insert data for 100 students of the same branch, then the branch information will be repeated for all those 100 students.
- These scenarios are nothing but **Insertion anomalies**.
- If you have to repeat the same data in every row of data, it's better to **keep the data separately** and **reference that data** in each row.
- So in the above table, we can keep the branch information separately, and just use the **branch_id** in the student table, where **branch_id** can be used to get the branch information.

2. Updation Anomaly in DBMS

- What if Mr. X leaves the college? or Mr. X is no longer the HOD of the computer science department? In that case, all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency.

- This is an Updation anomaly because you need to update all the records in your table just because one piece of information got changed.

3. Deletion Anomaly in DBMS

- In our **Student** table, two different pieces of information are kept together, the **Student information** and the **Branch information**.

- So if only a single student is enrolled in a branch, and that student leaves the college, or for some reason, the entry for the student is deleted, we will lose the branch information too.

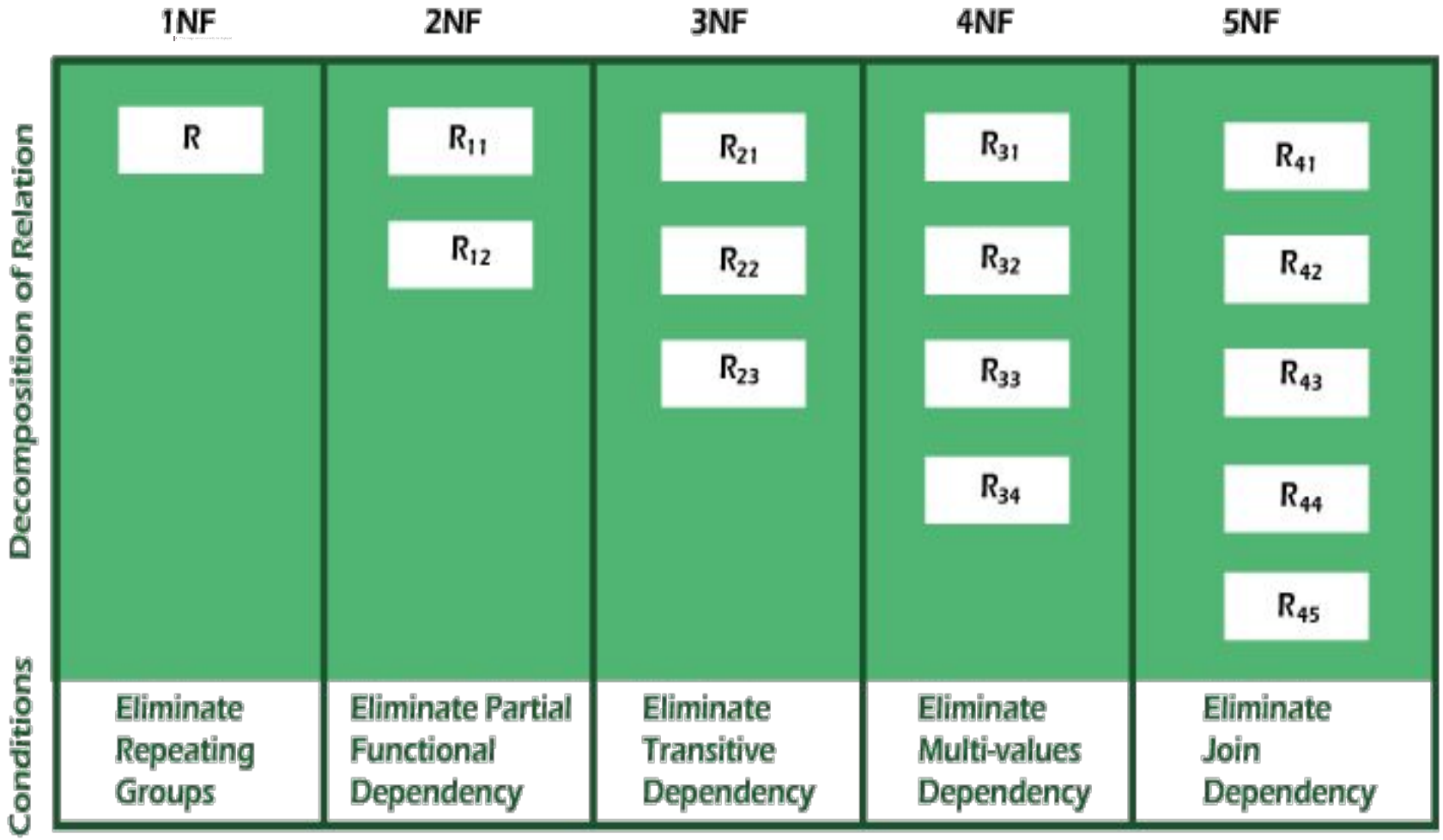
- So never in DBMS, we should keep two different entities together, which in the above example is Student and branch,

Types of DBMS Normal forms

Normalization rules are divided into the following normal forms:

1. First Normal Form
2. Second Normal Form
3. Third Normal Form
4. BCNF
5. Fourth Normal Form
6. Fifth Normal Form

Types of DBMS Normal forms



Types of DBMS Normal forms

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Types of DBMS Normal forms

1. First Normal Form (1NF)

For a table to be in the First Normal Form, it should follow the following 4 rules:

1. It should only have single(**atomic**) valued attributes/columns.
2. Values stored in a column should be of the same domain.
3. All the columns in a table should have unique names.
4. And the order in which data is stored should not matter.

If we have an **Employee** table in which we store the *employee information* along with the *employee skillset*, the table will look like this:

emp_id	emp_name	emp_mobile	emp_skills
1	John Tick	9999957773	Python, JavaScript
2	Darth Trader	8888853337	HTML, CSS, JavaScript
3	Rony Shark	7777720008	Java, Linux, C++

1. Create Separate tables for Employee and Employee Skills

So the **Employee** table will look like this,

emp_id	emp_name	emp_mobile
1	John Tick	9999957773
2	Darth Trader	8888853337
3	Rony Shark	7777720008

And the new **Employee_Skill** table:

emp_id	emp_skill
1	Python
1	JavaScript
2	HTML
2	CSS

2. Add Multiple rows for Multiple skills

You can also *simply add multiple rows* to add multiple skills. This will lead to repetition of the data, but that can be handled as you further Normalize your data using the Second Normal form and the Third Normal form.

emp_id	emp_name	emp_mobile	emp_skill
1	John Tick	9999957773	Python
1	John Tick	9999957773	JavaScript
2	Darth Trader	8888853337	HTML
2	Darth Trader	8888853337	CSS
2	Darth Trader	8888853337	JavaScript
3	Rony Shark	7777720008	Java
3	Rony Shark	7777720008	Linux
3	Rony Shark	7777720008	C++

What is Second Normal Form?

For a table to be in the Second Normal Form, it must satisfy two conditions:

- The table should be in the First Normal Form.
- There should be no Partial Dependency.

What is Dependency?

Let's take an example of a **Student** table with columns `student_id`, `name`, `reg_no`(registration number), `branch` and `address`(student's home address).

student_id	name	reg_no	branch	address

In this table, `student_id` is the primary key and will be unique for every row, hence we can use `student_id` to fetch any row of data from this table

Even for a case, where student names are same, if we know the `student_id` we can easily fetch the correct record.

student_id	name	reg_no	branch	address
10	Akon	07-WY	CSE	Kerala
11	Akon	08-WY	IT	Gujarat

Hence we can say a **Primary Key** for a table is the column or a group of columns(composite key) which can uniquely identify each record in the table.

What is Partial Dependency?

Let's create another table for **Subject**, which will have `subject_id` and `subject_name` fields and `subject_id` will be the primary key.

subject_id	subject_name
1	Java
2	C++
3	Php

Let's create another table **Score**, to store the **marks** obtained by students in the respective subjects. We will also be saving **name of the teacher** who teaches that subject along with marks.

score_id	student_id	subject_id	marks	teacher
1	10	1	70	Java Teacher
2	10	2	75	C++ Teacher
3	11	1	80	Java Teacher

In the score table we are saving the **student_id** to know which student's marks are these and **subject_id** to know for which subject the marks are for.

Together, `student_id + subject_id` forms a **Candidate Key**(learn about Database Keys) for this table, which can be the **Primary key**.

Confused, How this combination can be a primary key?

See, if I ask you to get me marks of student with `student_id` 10, can you get it from this table? No, because you don't know for which subject. And if I give you `subject_id`, you would not know for which student. Hence we need `student_id + subject_id` to uniquely identify any row.

But where is Partial Dependency?

Now if you look at the **Score** table, we have a column names `teacher` which is only dependent on the subject, for Java it's Java Teacher and for C++ it's C++ Teacher & so on.

Now as we just discussed that the primary key for this table is a composition of two columns which is `student_id` & `subject_id` but the teacher's name only depends on subject, hence the `subject_id`, and has nothing to do with `student_id`.

This is **Partial Dependency**, where an attribute in a table depends on only a part of the primary key and not on the whole key.

How to remove Partial Dependency?

The simplest solution is to remove columns `teacher` from Score table and add it to the Subject table. Hence, the Subject table will become:

subject_id	subject_name	teacher
1	Java	Java Teacher
2	C++	C++ Teacher
3	Php	Php Teacher

And our Score table is now in the second normal form, with no partial dependency.

score_id	student_id	subject_id	marks
1	10	1	70
2	10	2	75
3	11	1	80

How to remove Partial Dependency?

The simplest solution is to remove columns `teacher` from Score table and add it to the Subject table. Hence, the Subject table will become:

subject_id	subject_name	teacher
1	Java	Java Teacher
2	C++	C++ Teacher
3	Php	Php Teacher

And our Score table is now in the second normal form, with no partial dependency.

score_id	student_id	subject_id	marks
1	10	1	70
2	10	2	75
3	11	1	80

What is Third Normal Form?

Third Normal Form is an upgrade to Second Normal Form. When a table is in the Second Normal Form and has no transitive dependency, then it is in the Third Normal Form.

Student Table

student_id	name	reg_no	branch	address
10	Akon	07-WY	CSE	Kerala
11	Akon	08-WY	IT	Gujarat
12	Bkon	09-WY	IT	Rajasthan

Subject Table

subject_id	subject_name	teacher
1	Java	Java Teacher
2	C++	C++ Teacher
3	Php	Php Teacher

Score Table

score_id	student_id	subject_id	marks
1	10	1	70
2	10	2	75

Score Table

score_id	student_id	subject_id	marks
1	10	1	70
2	10	2	75
3	11	1	80

In the Score table, we need to store some more information, which is the exam name and total marks, so let's add 2 more columns to the Score table.

score_id	student_id	subject_id	marks	exam_name	total_marks

What is Transitive Dependency?

When a non-prime attribute depends on other non-prime attributes rather than depending upon the prime attributes or primary key.

How to remove Transitive Dependency?

Again the solution is very simple. Take out the columns `exam_name` and `total_marks` from Score table and put them in an **Exam** table and use the `exam_id` wherever required.

Score Table: In 3rd Normal Form

score_id	student_id	subject_id	marks	exam_id

The new Exam table

exam_id	exam_name	total_marks
1	Workshop	200
2	Mains	70
3	Practicals	30

Boyce-Codd Normal Form (BCNF)

Rules for BCNF

For a table to satisfy the Boyce-Codd Normal Form, it should satisfy the following two conditions:

1. It should be in the **Third Normal Form**.
2. And, for any dependency A → B, A should be a **super key**.

The second point sounds a bit tricky, right? In simple words, it means, that for a dependency A → B, A cannot be a **non-prime attribute**, if B is a **prime attribute**.

Boyce-Codd Normal Form (BCNF)

student_id	subject	professor
101	Java	P.Java
101	C++	P.Cpp
102	Java	P.Java2
103	C#	P.Chash
104	Java	P.Java

In the table above:

- One student can enrol for multiple subjects. For example, student with **student_id** 101, has opted for subjects - Java & C++
- For each subject, a professor is assigned to the student.
- And, there can be multiple professors teaching one subject like we have for Java.

Boyce-Codd Normal Form (BCNF)

What do you think should be the **Primary Key**?

Well, in the table above `student_id, subject` together form the primary key, because using `student_id` and `subject`, we can find all the columns of the table.

One more important point to note here is, one professor teaches only one subject, but one subject may have two different professors.

Hence, there is a dependency between `subject` and `professor` here, where `subject` depends on the professor name.

Why this table is not in BCNF?

In the table above, `student_id, subject` form primary key, which means `subject` column is a **prime attribute**

But, there is one more dependency, `professor` → `subject`.

And while `subject` is a prime attribute, `professor` is a **non-prime attribute**, which is not allowed by BCNF.

How to satisfy BCNF?

To make this relation(table) satisfy BCNF, we will decompose this table into two tables, **student** table and **professor** table.

Below we have the structure for both the tables.

Student Table

student_id	**p_id**
101	1
101	2
and so on...	

And, **Professor Table**

p_id	**professor**	**subject**
1	P.Java	Java
2	P.Cpp	C++
and so on...		

Fourth Normal Form (4NF)

Rules for 4th Normal Form

For a table to satisfy the Fourth Normal Form, it should satisfy the following two conditions:

1. It should be in the **Boyce-Codd Normal Form**.
2. And, the table should not have any **Multi-valued Dependency**.

What is Multi-valued Dependency?

A table is said to have multi-valued dependency, if the following conditions are true,

1. For a dependency A → B, if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.
2. Also, a table should have at-least 3 columns for it to have a multi-valued dependency.
3. And, for a relation `R(A,B,C)`, if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.

If all these conditions are true for any relation(table), it is said to have multi-valued dependency.

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
2	C#	Cricket
2	Php	Hockey

As you can see in the table above, student with `s_id` **1** has opted for two courses, **Science** and **Maths**, and has two hobbies, **Cricket** and **Hockey**.

You must be thinking what problem this can lead to, right?

Well the two records for student with `s_id` **1**, will give rise to two more records, as shown below, because for one student, two hobbies exists, hence along with both the courses, these hobbies should be specified.

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
1	Science	Hockey
1	Maths	Cricket

How to satisfy 4th Normal Form?

To make the above relation satify the 4th normal form, we can decompose the table into 2 tables.

CourseOpted Table

s_id	course
1	Science
1	Maths
2	C#
2	Php

And, **Hobbies Table**,

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

Database Design Process

1. Requirements Gathering

- **Identify Stakeholders**: Meet with users, developers, and other stakeholders to understand their needs.
- **Define Objectives**: Clarify what the database should accomplish.

2. Conceptual Design

- **Create an Entity-Relationship Diagram (ERD)**: Identify entities, attributes, and relationships between entities.
- **Define Data Models**: Use high-level data models to visualize the structure.

3. Logical Design

- **Choose a Database Model**: Decide on a relational, NoSQL, or other database models based on requirements.
- **Normalization**: Organize data to reduce redundancy and improve integrity.
- **Define Schemas**: Establish a logical schema, including tables, fields, and relationships.

Database Design Process

4. Physical Design

- **Select a Database Management System (DBMS)**: Choose a suitable DBMS based on scalability, performance, and requirements.
- **Define Storage Structure**: Determine how data will be stored (e.g., indexing strategies, partitions).
- **Design Security**: Implement access controls and data protection measures.

5. Implementation

- **Create the Database**: Set up the database structure using the chosen DBMS.
- **Data Migration**: If applicable, transfer existing data into the new database.
- **Develop Applications**: Create any applications that will interact with the database.

6. Testing

- **Performance Testing**: Check for speed and responsiveness under load.
- **Validation**: Ensure data integrity and accuracy through testing.
- **User Acceptance Testing (UAT)**: Get feedback from stakeholders and end-users.

Database Design Process

7. Deployment

- **Launch the Database**: Move the database into production.
- **Monitor Performance**: Continuously track performance and make adjustments as needed.

8. Maintenance and Optimization

- **Regular Backups**: Implement a backup strategy to prevent data loss.
- **Monitor Usage**: Analyze usage patterns for optimization.
- **Update and Scale**: Adjust the design and scale the database as user needs grow.

Functional Dependency

The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table.

> $X \rightarrow Y$

The left side of FD is known as a determinant, the right side of the production is known as a dependent.

Functional dependency can be written as:

> Emp_Id → Emp_Name

Types of Functional Dependencies

1. **Trivial Functional Dependency:**
   - A trivial functional dependency is one where the dependent set is a subset of the determinant set.
   - Example:
     - **{A, B} → A** or **{A, B} → B**.
     - Here, the right-hand side is always a part of the left-hand side, so it's a trivial dependency.
2. **Non-Trivial Functional Dependency:**
   - A non-trivial FD occurs when the dependent set is not a subset of the determinant set.
   - Example:
     - **A → B**.
     - This means that the value of attribute **A** determines the value of **B**, and **A** is not a part of **B**.

Types of Functional Dependencies

3. **Transitive Functional Dependency:**
   - A transitive FD happens when **A** → **B** and **B** → **C** imply **A** → **C**.
   - This means that there is an indirect dependency, and the determinant indirectly determines the dependent attribute via another attribute.
   - Example:
     - If **A** → **B** and **B** → **C**, then **A** → **C** (transitive).
4. **Multivalued Dependency (MVD):**
   - A multivalued dependency is a special type of functional dependency where one attribute or a set of attributes determines a set of values for another attribute or set of attributes.
   - Example:
     - **A** →→ **B**.
     - This means that for each value of **A**, there is a set of corresponding values for **B**. It is different from functional dependency because here, the number of values of **B** is not determined by the number of values of **A**.

Normalization videos Link